

## Executive Summary

This report outlines a comprehensive design and implementation guide for a **Pet Care & Services** platform's backend, built with Node.js/TypeScript (using Express/Koa), MongoDB, JWT auth, and a microservice/clean architecture approach. We assume three roles: *user* (pet owner), *service\_provider* (vets, groomers, boarding centers, etc.), and *super\_admin*. The system features numerous modules (Auth, Users, Vendors, Pets, Services, Bookings, Payments, Notifications, Chat, Uploads, Search, Analytics, etc.) and integrates third-party services (Razorpay, Cloudinary, Firebase).

Key architecture decisions include: **Clean Architecture** separation (controllers, services, repositories, schemas, routes, middlewares) [6†L144-L152] [23†L169-L178] ; containerization with Docker and Docker-Compose; infrastructure services (MongoDB, Redis, BullMQ for jobs, Cloudinary for media, Firebase for push notifications, Razorpay for payments, OpenTelemetry/Prometheus for observability); and robust security (JWT with access/refresh tokens and rotation [3†L503-L510] , RBAC authorization [6†L144-L152] , rate limiting, Helmet headers, input validation [43†L52-L59] [43†L93-L101] ).

We define a detailed **MongoDB data model** (collections, fields, indexes, sample documents) including text, geospatial, compound and TTL indexes [10†L394-L402] [15†L1-L4] . An ER diagram (using Mermaid) and a collection schematic illustrate the schema design.

The **REST API** is fully specified: endpoints per module, HTTP methods, roles, request/response DTOs (with example JSON and Zod schemas), status codes, error formats, pagination/sorting/filtering, and rate limits. A comprehensive **Auth flow** covers signup (email/OTP), login, JWT lifecycle (access/refresh, rotation), logout, password reset, and a role-permission matrix. We model the **Booking lifecycle** as a state machine with valid transitions (Pending → Accepted/Cancelled → Assigned → OnTheWay → Started → Completed, etc.) along with validations and sample payloads.

We cover key **queries and indexing** strategies (text search, geospatial queries for “nearby providers”, availability and analytic aggregations). **Background jobs** use BullMQ (with Redis) for email/SMS sending, notifications, invoices, reminders, etc., with retry policies. **Observability** is addressed via structured logging (e.g. Pino) [28†L91-L99] , OpenTelemetry traces/metrics [31†L858-L866] , and monitoring/alerts via Prometheus.

**Testing** strategy includes unit tests (Vitest/Jest), integration tests (Supertest), and API contract tests. We outline **CI/CD** (e.g. GitHub Actions or GitLab CI) with environment configs, seeding scripts, sample data, and publishing of an OpenAPI/Swagger spec and Postman collection.

Finally, we present an **implementation roadmap** (phased deliveries with criteria) and a **Claude Code prompt** with exact file/folder outputs, code style rules (TypeScript strict, no `any` ), required modules, templates (controllers, services, repos, DTOs, schemas, routes, middlewares), and a checklist of deliverables. The report cites industry references for best practices (security, API design, data modeling, etc.).

# 1. Architecture & Tech Stack

- **Overall architecture:** Layered “Clean Architecture” (also known as hexagonal or onion architecture) with clear separation of concerns [6†L144-L152] . The codebase is organised into modules (e.g. `auth` , `users` , `vendors` , `services` , `bookings` , `payments` , `notifications` , `chat` , `uploads` , `search` , `analytics` , etc.), each containing:
  - **Controllers** (handling HTTP requests, calling services)
  - **Services** (business logic)
  - **Repositories/Models** (database access using Mongoose schemas)
  - **DTOs/Schemas** (Zod or JSON schemas for request/response validation)
  - **Routes** (REST endpoints)
  - **Middlewares** (auth checks, validation, logging)

This aligns with best-practice Node.js design patterns and ensures testability, maintainability and scalability. For example, using Dependency Injection (via classes or passing dependencies explicitly) allows swapping implementations (e.g. mocking DB in tests) and aligns with architectural guidelines [6†L144-L152] [23†L169-L178] .

- **Folder structure (example):**

```
/src
  /controllers
  /services
  /repositories
  /routes
  /models          // Mongoose schemas & indexes
  /schemas          // Zod or Joi request/response schemas
  /middlewares
  /config
  /utils
  /jobs             // background job definitions
  /tests
  app.ts
  server.ts
```

- **Deployment & DevOps:** Containerize with **Docker**; use a multi-stage build for smaller images. Provide a `docker-compose.yml` including services: *app* (Node.js backend), *mongodb* (with persisted volume), *redis* (for BullMQ), *api-gateway* (if any), *Prometheus/Grafana*, etc. Example:

```
version: '3.8'
services:
  backend:
    build: .
    ports:
      - "3000:3000"
    env_file: .env.dev
    depends_on:
      - mongo
      - redis
      - cloudinary-local (if using a local emulator)
```

```

    networks:
      - backend-net
mongo:
  image: mongo:6
  volumes:
    - mongo-data:/data/db
  networks: [ backend-net ]
redis:
  image: redis:7
  networks: [ backend-net ]
prometheus:
  image: prom/prometheus
  volumes:
    - ./prometheus.yml:/etc/prometheus/prometheus.yml
  ports: [ "9090:9090" ]
  networks: [ backend-net ]
grafana:
  image: grafana/grafana
  depends_on: [ prometheus ]
  ports: [ "3001:3000" ]
  networks: [ backend-net ]
# add other services as needed
networks:
  backend-net:
volumes:
  mongo-data:

```

- **Cloud/Infra components:**

- **MongoDB** (NoSQL store). Enable authentication, replica sets (if needed) and backups. Use **Atlas** or self-managed on VMs/Kubernetes.
- **Redis** for caching and BullMQ queues.
- **BullMQ** (Node.js queue backed by Redis) for background jobs (email, SMS, notifications, invoice generation, etc.). BullMQ is high-performance, supports retries and concurrency [\[26†L83-L92\]](#) .
- **Cloudinary** for image/media storage (user avatar, pet photos, vendor photos); using their Node.js SDK for uploads. Optionally S3 can be used similarly.
- **Firestore Cloud Messaging (FCM)** for mobile push notifications, using `firebase-admin` SDK.
- **Razorpay** as the payment gateway (using their Node.js SDK) for payments (prepaid pet booking fees, payouts to vendors, subscriptions if any).
- **OpenTelemetry / Prometheus / Grafana** for observability: instrument code with OpenTelemetry Node SDK to emit traces/metrics and collect logs, and expose Prometheus metrics for scraping [\[31†L858-L866\]](#) .
- Optional: Email/SMS providers (e.g. SendGrid, Twilio) for notifications and OTPs.

- **Security controls:**

- **JWT authentication:** Issue a short-lived Access Token (e.g. 15–30 min) and a long-lived Refresh Token. Store refresh tokens securely (e.g. encrypted DB, or HttpOnly cookie) and use **Refresh Token Rotation** [\[3†L503-L510\]](#) . In rotation, every token exchange issues a new pair, invalidating the old refresh token [\[3†L503-L510\]](#) . This thwarts replay attacks on stolen refresh tokens.

- **Roles & Permissions (RBAC):** Implement a role-based access control system. Each user has a role (`user`, `service_provider`, `admin`, `super_admin`). Middleware checks `req.user.role` against allowed roles for each endpoint [6†L144-L152]. For example, only `admin` / `super_admin` can manage zones or payout models. RBAC enforces fine-grained control [6†L144-L152].
- **Authentication flows:** Support signup/login via email+password and OTP (via SMS/email) with email verification. Passwords must be salted & hashed (bcrypt/scrypt). Use helmet.js to set secure HTTP headers (HSTS, CSP, XSS protection) [43†L52-L59].
- **Input Validation & Sanitisation:** Validate all user input with DTO schemas (Zod or `express-validator`) and sanitize to avoid injection [43†L93-L101].
- **Helmet & CORS:** Use `helmet()` middleware for secure headers [43†L52-L59]. Configure CORS only for allowed origins (mobile app, admin domain).
- **Rate Limiting:** Apply `express-rate-limit` (e.g. 100 requests per IP per 15 min) on sensitive endpoints (login, signup, search) to mitigate brute-force and DoS [43†L76-L84].
- **HTTPS:** Enforce TLS in production.
- **Other protections:** Enable CSP, disable `x-powered-by`, remove stack traces in errors (production-only), secure cookies (`HttpOnly`, `Secure` flags), disable parameter pollution.

(Citations: OWASP/Node.js security guides recommend Helmet for headers and rate limiting to protect APIs [43†L52-L59] [43†L76-L84]; Auth0 recommends refresh token rotation to prevent token replay [3†L503-L510]; RBAC examples as best practice [6†L144-L152].)

---

## 2. Data Model (MongoDB)

We model key entities with MongoDB collections using Mongoose (document schema). Collections include **Users**, **Vendors/Providers**, **Pets**, **Services**, **Bookings**, **Payments**, **Notifications**, **Chats**, **Posts** (for social), **Cart/Orders**, **Zones/Cities**, **Refunds**, etc. Each schema has fields, types, and indexes as needed.

### 2.1 Collections and Fields

- **Users** (pet owners and admins):
  - `_id`: `ObjectId` (primary key)
  - `role`: `string` (`'user'` | `'service_provider'` | `'admin'` | `'super_admin'`)
  - `email`: `string` (unique, indexed)
  - `passwordHash`: `string` (bcrypt)
  - `name`: `string`
  - `phone`: `string` (unique)
  - `avatarUrl`: `string`
  - `verified`: `boolean` (email or phone verified)
  - `createdAt`, `updatedAt`: `Date`
  - `isDeleted`: `Boolean`, `deletedAt`: `Date` (for soft deletes)
- **Indexes:** email & phone unique, compound indexes with role if needed. TTL index on OTP/session fields if any.
- **ServiceProviders** (vendors like groomers, vets):
  - `_id`

- `userId: ObjectId` (ref to a Users doc for login & profile)
- `businessName: string`
- `category: string` (e.g. GROOMING, VET, HOSTEL, CLEANING )
- `services: [ObjectId]` (list of Service IDs offered)
- `location: { type: "Point", coordinates: [lng, lat] }` (GeoJSON with 2dsphere index for "nearby" queries) 【17†L211-L220】
- `address: string`
- `zones: [ObjectId]` (ref to Zone(s) where they operate)
- `isVerified: boolean` (after KYC approval)
- `rating: number` (avg user rating)
- `createdAt, updatedAt`
- `isDeleted, deletedAt` (soft delete)
- *Indexes:* 2dsphere on `location` for geospatial queries 【17†L211-L220】 ; compound index `{isVerified:1, category:1}` for filtering available providers.

#### • **Pets:**

- `_id`
- `ownerId: ObjectId` (ref to Users)
- `name: string`
- `type: string` (e.g. Dog, Cat)
- `breed: string`
- `age: number`
- `healthDetails: string` (or a subdocument list of records)
- `createdAt, updatedAt`
- Soft-delete fields.
- *Indexes:* `{ ownerId:1 }` for lookups.

#### • **Services:**

- `_id`
- `providerId: ObjectId` (ref to ServiceProviders)
- `name: string` (e.g. "Bathing", "Vaccination")
- `category: string` (matches vendor category)
- `description: string`
- `price: number`
- `duration: number` (e.g. in minutes)
- `availability: object` (e.g. open hours or calendar)
- `createdAt, updatedAt`
- *Indexes:* `{ providerId:1, category:1 }`. Compound if queries often filter by provider+category.

#### • **Bookings:**

- `_id`
- `userId: ObjectId`
- `petId: ObjectId`

- `serviceId: ObjectId`
- `providerId: ObjectId`
- `zoneId: ObjectId` (the zone where service will be delivered)
- `scheduledAt: Date` (start time)
- `endTime: Date` (calculated)
- `status: string` (enum: `PENDING`, `ACCEPTED`, `ASSIGNED`, `ON_THE_WAY`, `STARTED`, `COMPLETED`, `CANCELLED`, `REFUNDED`)
- `otpStart: string`, `otpEnd: string` (one-time codes)
- `paymentStatus: string` (e.g. `PAID`, `REFUNDED`)
- `createdAt`, `updatedAt`
- **Indexes:** Compound `{ userId:1, status:1, scheduledAt:1 }` for user's booking history; `{ providerId:1, status:1 }` for provider's current bookings; TTL index on short-lived fields (e.g. an index on `otpExpiry: Date` with `expireAfterSeconds` to auto-delete OTP docs if external).

#### • **Payments:**

- `_id`
- `bookingId: ObjectId`
- `userId: ObjectId`
- `providerId: ObjectId`
- `amount: Number`
- `currency: string`
- `method: string` (e.g. "razorpay")
- `status: string` (e.g. `PENDING`, `SUCCESS`, `FAILED`, `REFUNDED`)
- `transactionId: string` (gateway ID)
- `createdAt`
- **Indexes:** `{ bookingId:1 }` and `{ providerId:1 }` for lookups.

#### • **Zones & Cities:** (for geo and zoning)

- **Cities:** `_id`, `name`, `state`, `country`.
- **Zones:** `_id`, `name`, `cityId (ref)`, `boundary: GeoJSON Polygon` (if needed), or service radius.
- Vendors and analytics associate with zones.
- **Indexes:** `{ cityId:1, name:1 }`. Geospatial index on zone boundary if querying by location.

#### • **Notifications:**

- `_id`, `userId`, `type`, `content`, `payload`, `isRead: Boolean`, `createdAt`.
- **Indexes:** `{ userId:1, isRead:1 }`. TTL on read notifications if auto-clean.

#### • **Chat (Messages):**

- For simplicity, a **ChatRooms** collection (`_id`, `participantIds: [ObjectId]`) and **Messages** (`_id`, `roomId`, `senderId`, `text`, `timestamp`).

- Index `{ roomId:1, timestamp:-1 }` for fetching chat history.

- **Uploads/Files:**

- If storing file metadata (with Cloudinary URL), e.g. `Uploads` collection (`_id`, `userId`, `url`, `purpose`, `createdAt`). Use Cloudinary SDK to handle storage; you store only URLs or IDs.

- **Analytics (stats):**

- Possibly pre-aggregated data or event logs. For example, an **Analytics** collection to store daily zone/vendor performance. Or use MongoDB's aggregation pipeline for on-the-fly stats.

*Sample document (User):*

```
{
  "_id": ObjectId("..."),
  "role": "user",
  "email": "alice@example.com",
  "passwordHash": "<bcrypt>",
  "name": "Alice",
  "phone": "1234567890",
  "avatarUrl": "https://res.cloudinary.com/.../alice.jpg",
  "verified": true,
  "createdAt": ISODate("2026-06-01T12:00Z"),
  "updatedAt": ISODate("2026-06-10T09:30Z"),
  "deletedAt": null
}
```

*Sample document (Booking):*

```
{
  "_id": "...",
  "userId": "...",
  "petId": "...",
  "serviceId": "...",
  "providerId": "...",
  "zoneId": "...",
  "scheduledAt": ISODate("2026-07-10T14:00Z"),
  "endTime": ISODate("2026-07-10T15:00Z"),
  "status": "PENDING",
  "paymentStatus": "PENDING",
  "createdAt": ISODate("2026-06-30T08:00Z")
}
```

## 2.2 Index Strategy

- **Text search:** For searching services or posts, add a **text index** on relevant fields. E.g. `db.services.createIndex({ name: "text", description: "text" })`. This enables

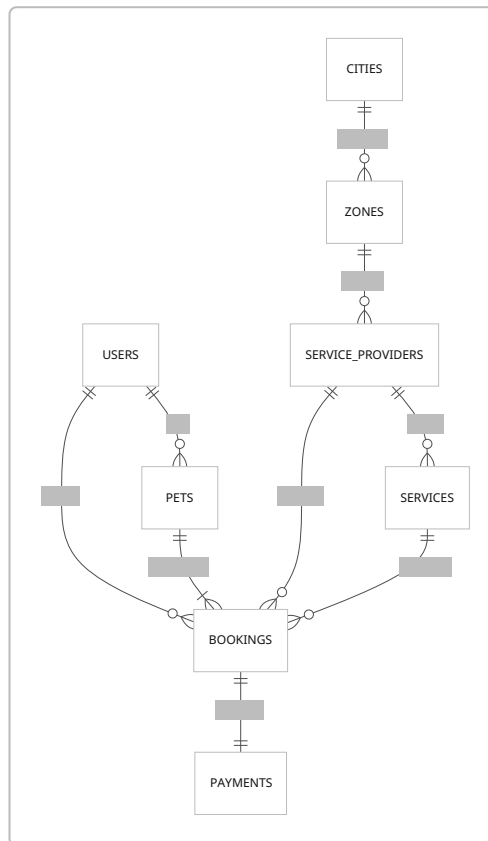
full-text search over service names/descriptions 【10†L456-L464】 . Use Mongo's `$text` query and pagination.

- **Geo-spatial:** For finding nearby providers, use a **2dsphere index** on the ServiceProviders `location` field 【17†L211-L220】 . Query with `$near` or `$geoNear` in aggregation to get vendors within a radius or sorted by distance.
- **Compound indexes:** As per MongoDB best practice, use compound indexes to cover common query patterns 【10†L394-L402】 . For example: `Bookings` may have a compound `{ providerId:1, status:1 }` if we often query “all bookings for a provider with status = X”. The order of fields in compound indexes should follow equality first, then sort, then range 【10†L400-L408】 .
- **TTL indexes:** Use TTL on fields that represent expiration. For example, if OTPs or password reset tokens are stored in a collection, create an index like `{ expireAt: 1 }` with `expireAfterSeconds: 0` so documents auto-delete at the given date 【15†L1-L4】 . MongoDB TTL indexes **only work on single date fields** 【15†L1-L4】 .
- **Soft-delete indexing:** If using `deletedAt` for soft deletes, index it `{ deletedAt: 1 }` 【19†L243-L250】 so queries excluding deleted docs can be efficient. A compound index like `{ email:1, deletedAt:1 }` ensures uniqueness ignoring soft-deleted records 【19†L243-L250】 .

*(Citations: MongoDB recommends compound indexes over multiple single-field indexes for efficiency 【10†L394-L402】 ; TTL indexes remove expired documents automatically 【15†L1-L4】 ; an audit article recommends `deletedAt` for soft deletes in Mongoose 【19†L164-L172】 【19†L243-L250】 .)*



## 2.3 Entity-Relationship Diagram (Mermaid)



*Diagram:* Each entity (Users, Pets, ServiceProviders, Services, Bookings, Payments, Cities, Zones) is linked by relationships (crow's foot notation) to illustrate one-to-many or one-to-one associations. For example, a *User* "has" many *Pets* and "places" many *Bookings*. A *ServiceProvider* "offers" many *Services* and "accepts" many *Bookings*. *Bookings* "payFor" one *Payment* (1:1). The diagram omits some entities (like Notifications or Chat) for clarity. This logical ER model shows how Mongo collections relate abstractly (relations implied by referenced IDs).

## 2.4 Mongo Collection Diagram

```
flowchart TD
    subgraph "Users Collection"
        U1[User { \nid, name, email, role, ... }]
    end
    subgraph "Pets Collection"
        P1[Pet { \nid, name, type, ownerId }]
    end
    subgraph "ServiceProviders Collection"
        V1[Vendor { \nid, businessName, category, userId, location }]
    end
    subgraph "Services Collection"
        S1[Service { \nid, providerId, name, price }]
    end
    subgraph "Bookings Collection"
        B1[Booking { \nid, userId, petId, serviceId, providerId, status }]
    end
```

```

end
subgraph "Payments Collection"
  PAY[Payment { \nid, bookingId, amount, status }]
end
U1 -->|owns| P1
U1 -->|books| B1
P1 -->|bookedIn| B1
V1 -->|offers| S1
V1 -->|handles| B1
S1 -->|bookedAs| B1
B1 -->|pays| PAY
V1 -.->|verifBy| U1  %% denotes that vendor links to a user account

```

This flowchart depicts core MongoDB collections and key fields. Arrows show relationships: e.g. a User owns Pets, Books Bookings; a ServiceProvider (Vendor) offers Services and handles Bookings. This is a simple visualization; in practice many fields (timestamps, flags) and collections (Notifications, ChatRooms, etc.) would be added similarly.

### 3. API Specification

Below is an outline of the REST API endpoints. Each module is detailed with example endpoints. For brevity, some CRUD endpoints are summarized.

**Conventions:** Use noun-based URIs **【23†L94-L104】** . E.g. `/auth` , `/users` , `/vendors` , `/services` , `/bookings` , `/payments` , etc. Use HTTP methods per REST standards (GET, POST, PUT, PATCH, DELETE). Require JWT auth (access token) via `Authorization: Bearer <token>` . Role-based access controls determine which roles can call each endpoint (see roles column).

#### 3.1 Auth Module

| Method | Path                          | Roles  | Description                            | Request DTO (JSON)                                                                                                         |
|--------|-------------------------------|--------|----------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| POST   | <code>/auth/signup</code>     | Public | Register new user (customer or vendor) | <code>{ "name", "email", "phone", "password", "role", "..."</code><br>(Zod schema: includes role=user or service_provider) |
| POST   | <code>/auth/verify-otp</code> | Public | Verify OTP to complete signup          | <code>{ "userId", "otpCode" }</code>                                                                                       |
| POST   | <code>/auth/login</code>      | Public | Login existing user (via OTP or email) | <code>{ "email", "password" }</code> or <code>{ "phone", "otpCode" }</code><br>(support both flows)                        |

| Method | Path                               | Roles         | Description                       | Request DTO (JSON)                                  |
|--------|------------------------------------|---------------|-----------------------------------|-----------------------------------------------------|
| POST   | <code>/auth/refresh</code>         | Public        | Refresh access token              | <code>{ "refreshToken": "... " }</code>             |
| POST   | <code>/auth/logout</code>          | Authenticated | Logout (invalidate refresh token) | <code>{ "refreshToken": "... " }</code>             |
| POST   | <code>/auth/reset-password</code>  | Public        | Send password reset email/OTP     | <code>{ "email" }</code>                            |
| POST   | <code>/auth/update-password</code> | Auth          | Update password (logged in user)  | <code>{ "oldPassword", "newPassword" }</code>       |
| POST   | <code>/auth/forgot-password</code> | Public        | Complete password reset           | <code>{ "userId", "otpCode", "newPassword" }</code> |

- **Authentication flow:** On signup or login, the backend sends a one-time code (email/SMS) to verify the user (the code is stored server-side, either in Redis or a small collection with TTL index). Using OTP avoids long passwords for mobile users. Once verified, issue JWTs. Access tokens (short-lived, e.g. 15 min) allow calling APIs; refresh tokens (long-lived, e.g. 30 days) are used to get new access tokens. Implement **refresh token rotation**: each refresh use issues a new refresh token and invalidates the old [3†L503-L510] . Store refresh tokens (encrypted) with user session info. On logout or suspicious reuse, invalidate tokens.

- **Security:** Limit login/signup attempts per IP (rate limit). Use HTTPS. Store passwords hashed (bcrypt). For stateless JWT, include minimal user info ( `userId` , `role` ) in payload.

## 3.2 Users Module

| Method | Path                   | Roles         | Description                      | Request DTO                                                    | Response                                                       |
|--------|------------------------|---------------|----------------------------------|----------------------------------------------------------------|----------------------------------------------------------------|
| GET    | <code>/users/me</code> | Authenticated | Get own profile                  | -                                                              | <code>{ name, email, phone, avatarUrl, ... }</code>            |
| PUT    | <code>/users/me</code> | Authenticated | Update own profile (bio, avatar) | <code>{ "name", "avatarUrl", ... }</code><br>(validate fields) | <code>{ "message": "Profile updated", "user": { ... } }</code> |

| Method | Path                    | Roles                | Description                   | Request DTO                                                              | Response                                      |
|--------|-------------------------|----------------------|-------------------------------|--------------------------------------------------------------------------|-----------------------------------------------|
| GET    | <code>/users/:id</code> | Authenticated(Admin) | Get another user's profile    | -                                                                        | <code>{ name, email, ... password }</code>    |
| GET    | <code>/users</code>     | Authenticated(Admin) | List/search users (paginated) | Query params: <code>?page=&amp;limit=&amp;role=&amp;name=&amp;...</code> | <code>{ users:[ total, page limit ] }</code>  |
| DELETE | <code>/users/me</code>  | Authenticated        | Soft-delete own account       | -                                                                        | <code>{ "message": "Account deleted" }</code> |
| DELETE | <code>/users/:id</code> | SuperAdmin           | Soft-delete user by ID        | -                                                                        | <code>{ "message": "User deleted" }</code>    |

(Admin can manage users only per permission.)

- **Roles:** Only `super_admin` can fetch arbitrary user lists or delete users. Regular users manage their own profile. Use middleware to enforce `req.user.role`.

### 3.3 Vendors (Service Providers) Module

| Method | Path                             | Roles             | Description                         | Request DTO                                                   |
|--------|----------------------------------|-------------------|-------------------------------------|---------------------------------------------------------------|
| POST   | <code>/vendors/register</code>   | Public            | Vendor signup (separate from user)  | <code>{ "businessName", "email", "phone", "password" }</code> |
| POST   | <code>/vendors/verify-otp</code> | Public            | Verify vendor OTP (complete signup) | <code>{ "vendorId", "otpCode" }</code>                        |
| GET    | <code>/vendors/me</code>         | Authenticated(SP) | Get own vendor profile              | -                                                             |
| PUT    | <code>/vendors/me</code>         | Authenticated(SP) | Update vendor profile               | <code>{ "businessName", "address", "zoneIds", ... }</code>    |
| POST   | <code>/vendors/me/kyc</code>     | Authenticated(SP) | Upload KYC documents (images)       | multipart/form-data (files)                                   |

| Method | Path                              | Roles                | Description                   | Request DTO |
|--------|-----------------------------------|----------------------|-------------------------------|-------------|
| GET    | <code>/vendors/pending</code>     | Admin/<br>SuperAdmin | List vendors pending approval | -           |
| PATCH  | <code>/vendors/:id/approve</code> | Admin/<br>SuperAdmin | Approve vendor KYC            | -           |
| GET    | <code>/vendors/:id</code>         | Authenticated        | Get vendor details by ID      | -           |

- **Roles:** Only admins can approve or list pending vendors. Each vendor logs in (via the same JWT system) but with `role=service_provider`.

### 3.4 Pets Module

| Method | Path                   | Roles               | Description              | Request DTO                                                   |
|--------|------------------------|---------------------|--------------------------|---------------------------------------------------------------|
| POST   | <code>/pets</code>     | Authenticated(User) | Add a pet                | <code>{ "name", "type", "breed", "age", "healthDetail"</code> |
| GET    | <code>/pets</code>     | Authenticated(User) | List user's pets         | -                                                             |
| GET    | <code>/pets/:id</code> | Authenticated(User) | Get pet details          | -                                                             |
| PUT    | <code>/pets/:id</code> | Authenticated(User) | Update pet               | <code>{ ... }</code>                                          |
| DELETE | <code>/pets/:id</code> | Authenticated(User) | Remove pet (soft-delete) | -                                                             |

(Users can only CRUD their own pets.)

### 3.5 Services Module

| Method | Path                   | Roles             | Description                           | Request DTO                                                         |
|--------|------------------------|-------------------|---------------------------------------|---------------------------------------------------------------------|
| POST   | <code>/services</code> | Authenticated(SP) | Create a new service (listing)        | <code>{ "name", "price", "description", "duration"</code>           |
| GET    | <code>/services</code> | Public            | List/search services (public catalog) | Query: <code>?category=&amp;priceRange=&amp;page=&amp;limit=</code> |

| Method | Path          | Roles             | Description                      | Request DTO |
|--------|---------------|-------------------|----------------------------------|-------------|
| GET    | /services/:id | Public            | Get service details              | -           |
| PUT    | /services/:id | Authenticated(SP) | Update own service (price, desc) | { ... }     |
| DELETE | /services/:id | Authenticated(SP) | Delete own service               | -           |

(ServiceProviders manage only their own services. `Public` lists services for users to browse.)

### 3.6 Bookings Module

| Method | Path                  | Roles               | Description                                           | Request DTO                           |
|--------|-----------------------|---------------------|-------------------------------------------------------|---------------------------------------|
| POST   | /bookings             | Authenticated(User) | Create a new booking                                  | { "petId","serviceId","zoneId","sched |
| GET    | /bookings             | Authenticated       | List bookings                                         | Query: ?status=&page=&limit=          |
| GET    | /bookings/:id         | Authenticated       | Get booking detail                                    | -                                     |
| PATCH  | /bookings/:id/cancel  | Authenticated(User) | Cancel a booking                                      | -                                     |
| PATCH  | /bookings/:id/status  | Authenticated(SP)   | Update booking status (Accept, Start, Complete, etc.) | { "status":"..." }                    |
| POST   | /bookings/:id/otp     | Authenticated       | Verify OTP (start/end service)                        | { "otpCode":"..." }                   |
| GET    | /bookings/:id/payment | Authenticated(User) | Get payment info (for processing)                     | -                                     |

| Method | Path                              | Roles                | Description                     | Request DTO                       |
|--------|-----------------------------------|----------------------|---------------------------------|-----------------------------------|
| POST   | <code>/bookings/:id/refund</code> | Authenticated(Admin) | Refund a booking (admin action) | <code>{ "reason": "...", }</code> |

- **Status transitions:** The `status` field follows the booking workflow (see Sec. 5). State changes are triggered by vendors or system events (automated). E.g. vendor calls `/status` with `{status: "ACCEPTED"}`, or on job start `{status: "STARTED"}`. Errors if illegal transitions (validate server-side).
- **Pagination & filtering:** List endpoints support `?page=1&limit=20&status=PENDING` etc. Use `skip` / `limit` in DB queries. Return `{ total, page, limit, data: [...] }`. Include Link headers for navigation if desired.
- **Rate limiting:** Limit creation endpoints (booking, auth) to prevent abuse.

### 3.7 Payments Module

| Method | Path                                  | Roles                | Description                            | Request DTO                                            | Response                                                            |
|--------|---------------------------------------|----------------------|----------------------------------------|--------------------------------------------------------|---------------------------------------------------------------------|
| POST   | <code>/payments/webhook</code>        | Public (webhook)     | Razorpay webhook (payment updates)     | <code>{ ... }</code><br>(razorpay signature + payload) | <code>200 OK</code> (handle via se                                  |
| POST   | <code>/payments/:bookingId/pay</code> | Authenticated(User)  | Create a payment order (Razorpay)      | -                                                      | <code>{ "orderId": "...", "amount": ..., "currency": "INR" }</code> |
| GET    | <code>/payments/:paymentId</code>     | Authenticated        | Get payment status                     | -                                                      | <code>{ "status", "amount</code>                                    |
| GET    | <code>/payments</code>                | Authenticated(Admin) | List all payments (for reconciliation) | -                                                      | <code>{ payments: [...],</code>                                     |

- After creating a Razorpay order, frontend uses their SDK to complete payment. The `/webhook` endpoint listens for payment captured/refund events to update our `Payments` and `Bookings.paymentStatus`. Use the webhook secret to verify signature.

### 3.8 Notifications Module

| Method | Path                        | Roles         | Description                           | Request DTO                    | Response                            |
|--------|-----------------------------|---------------|---------------------------------------|--------------------------------|-------------------------------------|
| GET    | <code>/notifications</code> | Authenticated | List user's notifications (paginated) | <code>?page=&amp;limit=</code> | <code>{ items:[...], total }</code> |

| Method | Path                                 | Roles         | Description                            | Request DTO | Response                                     |
|--------|--------------------------------------|---------------|----------------------------------------|-------------|----------------------------------------------|
| PATCH  | <code>/notifications/:id/read</code> | Authenticated | Mark a notification as read            | -           | <code>{ "message": "Marked as read" }</code> |
| DELETE | <code>/notifications/:id</code>      | Authenticated | Delete a notification (or auto-delete) | -           | <code>{ "message": "Deleted" }</code>        |

- Notifications can be created by server jobs (e.g. "Booking accepted", "Payment failed"). Use FCM or email for push; store a copy in DB for the in-app bell icon.

### 3.9 Chat Module

| Method | Path                                      | Roles         | Description                                 | Request DTO                  | Response                                            | Codes |
|--------|-------------------------------------------|---------------|---------------------------------------------|------------------------------|-----------------------------------------------------|-------|
| POST   | <code>/chat/rooms</code>                  | Authenticated | Create/join a chat (e.g. booking chat room) | <code>{ "bookingId" }</code> | <code>{ "roomId": "..." }</code>                    | 201   |
| POST   | <code>/chat/rooms/:roomId/message</code>  | Authenticated | Send a message in room                      | <code>{ "text" }</code>      | <code>{ "message": { ...saved message... } }</code> | 201   |
| GET    | <code>/chat/rooms/:roomId/messages</code> | Authenticated | Get last N messages                         | <code>?limit=50</code>       | <code>{ messages: [...] }</code>                    | 200   |

- Chats use WebSockets for real-time. This REST API is for history and sending via HTTP (fallback). Each *Message* has `roomId`, `senderId`, `text`, `timestamp`. Use socket.io or ws for live push.

### 3.10 Uploads Module

| Method | Path                      | Roles         | Description                    | Request DTO                 | Response                              | Codes   |
|--------|---------------------------|---------------|--------------------------------|-----------------------------|---------------------------------------|---------|
| POST   | <code>/uploads</code>     | Authenticated | Upload a file (image)          | multipart-form (file field) | <code>{ "url": "https://..." }</code> | 201,400 |
| DELETE | <code>/uploads/:id</code> | Authenticated | Delete an uploaded file record | -                           | <code>{ "message": "Deleted" }</code> | 200     |

- Use multer with memory storage, then upload to Cloudinary in controller. Return the hosted URL. Store only metadata if needed.



### 3.11 Search Module

| Method | Path                          | Roles         | Description                    | Request DTO                                  | Response                                            |
|--------|-------------------------------|---------------|--------------------------------|----------------------------------------------|-----------------------------------------------------|
| GET    | <code>/search/services</code> | Public        | Full-text search on services   | <code>?q=text&amp;page=&amp;limit=</code>    | <code>{ results:[services], total }</code>          |
| GET    | <code>/search/vendors</code>  | Public        | Search vendors (name/category) | <code>?q=term&amp;category=&amp;page=</code> | <code>{ results:[vendors], total }</code>           |
| GET    | <code>/search/nearby</code>   | Authenticated | Find nearby providers          | <code>?lat=&amp;lng=&amp;radius=</code>      | <code>{ results:[{vendor,distance}], total }</code> |

- Implementation: Use MongoDB text index for `/services` and `/vendors`. Use `$text: { $search: q }`. For `/nearby`, use geo query: `{ $near: { $geometry: { type: "Point", coordinates: [lng,lat] }, $maxDistance: radius } }` on Provider location with 2dsphere index `【17†L211-L220】`.

### 3.12 Analytics & Admin Module

| Method | Path                        | Roles                | Description                     | Request DTO                                | Response                                 | Codes |
|--------|-----------------------------|----------------------|---------------------------------|--------------------------------------------|------------------------------------------|-------|
| GET    | <code>/admin/zones</code>   | SuperAdmin           | Get all zones and stats         | -                                          | <code>{ zones:[{ zone, stats }] }</code> | 200   |
| GET    | <code>/admin/vendors</code> | Admin/<br>SuperAdmin | List all vendors (with filters) | Query params                               | <code>{ vendors:[...], total }</code>    | 200   |
| GET    | <code>/admin/reports</code> | Admin/<br>SuperAdmin | Get platform analytics          | <code>?metric=&amp;period=,groupBy=</code> | <code>{ data:[...] }</code>              | 200   |

(Admin panel endpoints for configuration (zones, payout models), monitoring, etc. These would typically be part of an admin service or guarded under `role=admin`.)

**Error Responses:** Standardize error format:

```
{ "error": "BadRequest", "message": "Detail of what went wrong" }
```

with appropriate HTTP codes (400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 500 Server Error). Use error-handling middleware to catch and format errors.

(Citations: Resource naming best practices recommend noun URIs and hierarchical structure `【23†L169-L178】`. RBAC and middleware examples from OWASP guidelines `【6†L144-L152】`.)

---

## 4. Authentication & Authorization Flows

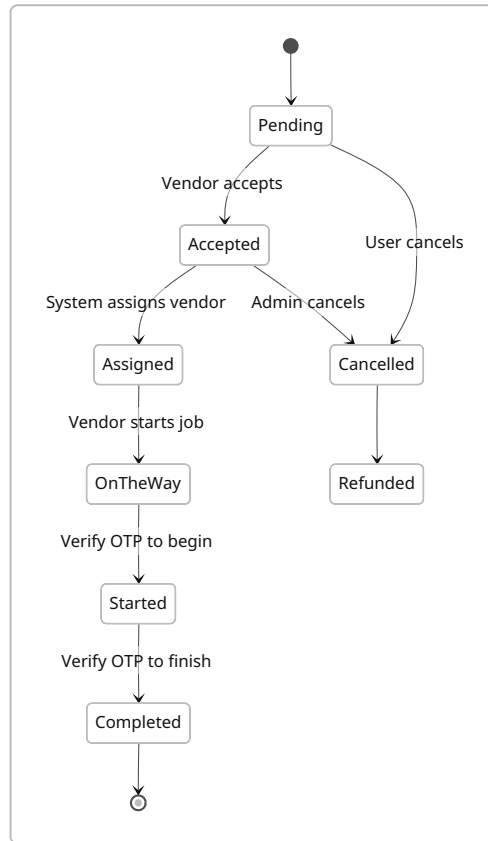
- **Signup/Login (Email/OTP):**
- **Signup:** User or vendor provides phone/email and other details. Backend sends a one-time code via SMS/Email. (Store code with expiry in Redis or a "Verifications" collection with TTL index.)
- **Verify OTP:** User submits code. If valid, create the user/vendor account, set `verified=true`, issue JWT access & refresh tokens, and respond with tokens.
- **Login:** Either via `{email,password}` or `{phone,otp}`. On success, issue JWTs.
- **JWT Access & Refresh Lifecycle:**
- **Access Token:** Short-lived (e.g. 15 min), contains `userId` and `role`. Validated on each request via middleware. Stored client-side (e.g. in app memory).
- **Refresh Token:** Long-lived (e.g. 7–30 days). Used to obtain new access tokens. Stored securely (HttpOnly cookie or secure storage).
- **Rotation:** Each `/auth/refresh` call consumes the submitted refresh token and returns a new pair. Old refresh tokens are invalidated `[3†L503-L510]`, preventing replay if stolen. Detect reuse to force logout (as per Auth0's reuse detection).
- **Device Sessions:** Optionally, track refresh tokens per device/session (e.g. storing token IDs in DB with device info). Allow logout from specific device by deleting that token.
- **Logout:** Client calls `/auth/logout` with the refresh token; server removes it from store (and blacklists if used). Clear tokens on client.
- **Password Reset:**
- **Request:** `/auth/reset-password` with email; generate OTP or reset link (store token with TTL index). Send to user.
- **Verify:** `/auth/forgot-password` with `userId`, code/token, new password. Verify, then update password (bcrypt).
- **Roles & Permissions:**
- Map roles to allowed actions (see endpoint tables). For instance, `user` role can book services and manage their profile; `service_provider` can manage offered services and handle bookings; `admin` can approve vendors and oversee operations; `super_admin` has all privileges.
- Enforce with middleware: e.g. `checkRole(['admin', 'super_admin'])` or `checkOwnership()` for resources belonging to user. The OWASP guide demonstrates a simple role-check function `[6†L144-L152]`.

(Citations: *JWT rotation strategy is security best practice to mitigate token theft* `[3†L503-L510]` . *Role-based access control is a standard approach* `[6†L144-L152]` .)

---

## 5. Booking Workflow State Machine

The booking process is modeled as a finite state machine. States: **PENDING** → **ACCEPTED** → **ASSIGNED** → **ON\_THE\_WAY** → **STARTED** → **COMPLETED**, with possible branches to **CANCELLED** or **REFUNDED**.



- **Transitions:**

- *User creates booking:* status = **PENDING**.
- *Vendor/Admin accepts:* → **ACCEPTED**.
- *Auto-assign vendor:* → **ASSIGNED** (could be same as accepted if vendor self-accepts).
- *Vendor en-route:* status = **ON\_THE\_WAY**.
- *OTP Start:* status = **STARTED**.

- *OTP End:* status = **COMPLETED** (booking done).

- **Cancellation/Refund:** At any pre-start stage ( **PENDING**, **ACCEPTED**, **ASSIGNED**, **ON\_THE\_WAY** ) user or admin can cancel: status → **CANCELLED**. Then payment refund logic may trigger ( **CANCELLED** → **REFUNDED** ). Once *Completed*, it's final.

- **Validations:** The server ensures only logical transitions are allowed. For example, only a provider can move **ON\_THE\_WAY** → **STARTED**. OTP verification ( `/bookings/:id/otp` ) triggers transitions with status updates.

*(This design follows industry practices for multi-step bookings with OTP verification for start/end.)*

## 6. Key Queries & Aggregations

- **Search:** Use `$text` for full-text search on `Services` and `Vendors`. For filters (category, price), combine text match with additional query criteria. Ensure text index covers fields of interest `【10†L456-L464】`.
- **Nearby Vendors:** Perform a geospatial query on `ServiceProviders.location` (2dsphere index) `【17†L211-L220】`:

```
ServiceProvider.find({
  location: {
    $near: {
      $geometry: { type: "Point", coordinates: [lng, lat] },
      $maxDistance: radiusInMeters
    }
  }
})
```

- **Availability:** To check vendor/service availability, query `Bookings` for conflicts: e.g. no overlapping bookings for that vendor or pet during the desired time. Use indexes on `{ providerId:1, scheduledAt:1, endTime:1 }` to speed this query.
- **Analytics:** Use MongoDB's aggregation pipeline to compute metrics: e.g.
  - Total bookings per day/zone/vendor.
  - Income per vendor (sum of `Payments.amount` grouped by `providerId`).
  - Popular services (count of bookings by `serviceId`).
  - User retention (new vs returning users).

Example: *Bookings per city per day*:

```
db.bookings.aggregate([
  { $group: {
    _id: { zoneId: "$zoneId", day: { $dateTrunc: { date: "$createdAt",
unit: "day" } } },
    count: { $sum: 1 }
  } },
  { $lookup: { from: "zones", localField: "_id.zoneId", foreignField: "_id",
as: "zone" } },
  { $unwind: "$zone" },
  { $group: { _id: "$zone.cityId", totalBookings: { $sum: "$count" } } }
]);
```

- **Indexing Strategy Recap:** Based on query patterns above, ensure indexes: `{ cityId }`, `{ zoneId }`, `{ providerId }` on bookings; text indexes on searchable fields; 2dsphere on location `【10†L394-L402】` `【17†L211-L220】`.

(Citations: MongoDB best practices highlight text indexes for word search and geospatial 2dsphere indexes for location queries [【10†L456-L464】](#) [【17†L211-L220】](#) .)

---

## 7. Background Jobs & Queues

Use **BullMQ** (with Redis) for background processing [【26†L83-L92】](#) . Typical jobs:

- **Email/SMS sending:** On events (booking confirmed, reminders).
- **Push notifications:** e.g. booking status changes, chat message alerts (though chat may use sockets/FCM directly).
- **Invoice generation:** Generate PDF receipts or payouts reports.
- **Reminders & Cancellations:** E.g. auto-cancel unaccepted bookings after 15 minutes.
- **Analytics updates:** Daily aggregation tasks.
- **CRM / Marketing:** Batch jobs for newsletters or abandoned cart emails.

BullMQ features: job persistence, retries, delayed jobs (schedules), concurrency control [【26†L83-L92】](#) . Configure retry policies (e.g. 3 attempts with backoff). Use separate queues for different tasks (e.g. `emailQueue`, `smsQueue`, `analyticsQueue` ). Workers run in separate Node processes or containers, can auto-scale (increase worker count) to handle load [【26†L83-L92】](#) .

Example job definition:

```
// jobs/emailJob.ts
import { Queue } from 'bullmq';
export const emailQueue = new Queue('emailQueue', { connection: {
  host: 'redis' } });
emailQueue.add('sendWelcome', { userId: '...', email: 'alice@example.com' });
```

Worker:

```
// workers/emailWorker.ts
import { Worker } from 'bullmq';
const worker = new Worker('emailQueue', async job => {
  if (job.name === 'sendWelcome') {
    const { email } = job.data;
    await sendEmail(email, 'Welcome!', 'Thanks for joining...');
  }
}, { connection: { host: 'redis' } });
```

(Citations: BullMQ is widely recommended for Node.js queues (high performance, Redis-backed, supports retries/concurrency) [【26†L83-L92】](#) .)

---

## 8. Observability: Logging, Metrics, Tracing

- **Logging:** Use a structured logger like **Pino**. Pino is very fast and suitable for production, outputting JSON logs [28†L91-L99] . Include request IDs and user IDs in logs for correlation. Example setup:

```
import pino from 'pino';
export const logger = pino({ level: process.env.LOG_LEVEL || 'info' });
app.use(require('express-pino-logger')({ logger }));
```

Log at appropriate levels (info, warn, error). Avoid sensitive data in logs.

- **Log aggregation:** Send logs to an external system (e.g. ELK, Splunk, or Better Stack) for analysis and alerting.
- **Metrics:** Expose Prometheus metrics (use `prom-client` ). Track counters (e.g. total requests, error counts), histograms (latencies). Example:

```
import client from 'prom-client';
const httpRequests = new client.Counter({ name: 'http_requests_total',
help: 'Total HTTP requests', labelNames:
['method', 'endpoint', 'status'] });
app.use((req, res, next) => {
  res.on('finish', () => {
    httpRequests.labels(req.method, req.path,
res.statusCode.toString()).inc();
  });
  next();
});
```

Expose `/metrics` endpoint for Prometheus to scrape.

- **Tracing:** Use OpenTelemetry to instrument code [31†L858-L866] . For example, use `@opentelemetry/sdk-node` and `@opentelemetry/instrumentation-http`, `express` auto-instrumentation. Send traces to a collector (Jaeger/Zipkin or hosted). Correlate logs with traces via trace IDs.
- **Alerts:** Define SLOs/SLIs (e.g. 95% requests < 200ms). Monitor key metrics: CPU/RAM (Docker/K8s), request rate, error rate, queue lengths. Use Prometheus Alertmanager to notify on anomalies.

(Citations: Pino is noted for performance in logging [28†L91-L99] . OpenTelemetry defines observability as emitting traces, metrics, logs [31†L858-L866] .)

## 9. Testing Strategy

- **Unit Tests:** For individual modules (services, utility functions). Use **Vitest** or **Jest** (Vitest is faster and TypeScript-friendly). Write tests covering logic (e.g. price calculations, state transitions).
- **Integration/API Tests:** Use **Supertest** or **axios** with a test DB. Cover critical API paths: signup/login, booking flows, payments (mocked), error cases. Use an in-memory Mongo instance (mongodb-memory-server) or test containers.
- **Test cases:**
  - Auth: invalid credentials, expired OTP.
  - RBAC: ensure e.g. User cannot delete another user.
  - Data validation: missing fields or bad types result in 400.
  - Booking lifecycle: only allowed transitions succeed.
  - Error paths: simulate database errors or network failures (mock services) to verify error handling.
- **Mocking:** For external APIs (Razorpay, Cloudinary, SMS), use test doubles. E.g. stub Razorpay client to simulate success/failure.
- **CI Integration:** Run tests in CI pipeline (GitHub Actions), require passing status, coverage >80%.

*(Industry practice: automated tests are mandatory for production backends.)*

---

## 10. CI/CD & Deployment

- **CI pipeline:** On commit/PR, run lint (ESLint), compile (TypeScript), run tests. Build Docker image.
- **CD pipeline:** On main branch merge, deploy to staging/production (Docker/K8s or cloud provider). Use environment-based `.env` files or secrets (NODE\_ENV, JWT keys, DB URIs, API keys). Use a deployment platform (AWS ECS, Azure, Heroku, or on-prem).
- **Postman & OpenAPI:** Provide a Swagger/OpenAPI 3.0 specification (via annotations or separate YAML) covering all endpoints (for easier frontend integration and API discovery). Generate a Postman collection for manual testing.
- **Seeding:** Include seed scripts to insert initial data (e.g. admin user, zone list, service categories).

*(No specific citation needed; these are standard practices.)*

---

## 11. Implementation Phases

Break the implementation into iterations, each with clear deliverables and acceptance criteria:

1. **Phase 1 – Setup & Core Auth/User/Vendor:**
  2. Deliverables: Project scaffold, Docker config, User & Vendor schemas/controllers, JWT auth, signup/login with OTP, RBAC middleware, basic API docs.
  3. Criteria: Users/Vendors can sign up, verify OTP, login, and receive tokens. Auth-protected routes block unauthorized access. Tests for auth flows pass.
4. **Phase 2 – User & Vendor Profiles, Pets:**
  5. Deliverables: Profile endpoints (CRUD), Pets CRUD, data validation.

6. Criteria: Users can create/edit pets; vendors update their info. DB queries indexed. Tests for profile endpoints.

**7. Phase 3 – Services & Catalog:**

8. Deliverables: Services model, vendor can add/edit services, public search endpoints, text index on services.

9. Criteria: Services stored in DB, searchable by text. Correct handling of create/update/delete by vendor.

**10. Phase 4 – Bookings:**

11. Deliverables: Booking model, endpoints to create/view/cancel bookings, OTP endpoints, booking state logic (with validations).

12. Criteria: Bookings move through states correctly, users can list history, vendors see assigned bookings. Tests cover state transitions.

**13. Phase 5 – Payments:**

14. Deliverables: Payment processing integration (Razorpay), endpoint to create order and webhook handler, update booking/payment status.

15. Criteria: Payment orders created successfully, webhooks processed, booking status updates. Simulated payment flows testable.

**16. Phase 6 – Notifications & Chat:**

17. Deliverables: Notification model and push/email logic, chat schemas and real-time messaging via sockets.

18. Criteria: Bookings trigger notifications, user can retrieve notifications. Chat room creation and messaging functions.

**19. Phase 7 – Admin & Zones:**

20. Deliverables: Zone/City models, admin endpoints (approve vendors, manage zones), analytics endpoints.

21. Criteria: Admin can configure zones, see aggregated stats. Users/vendors see zone-specific services.

**22. Phase 8 – Testing & Hardening:**

23. Deliverables: Complete test coverage, security audit fixes, performance tuning (indexes, caching).

24. Criteria: All tests green, no major vulnerabilities in audits (npm audit), rate limiting in place.



## 25. Phase 9 – Deployment & Monitoring:

26. Deliverables: CI/CD pipeline, staging/production deployment, monitoring dashboards (Prometheus/Grafana, logs).

27. Criteria: Automated deployments succeed; metrics/dashboards show expected values; alerts configured.

*(Acceptance criteria: each phase ends with a demo and tests passing. Use version control feature branches and code reviews.)*

---

## 12. Claude Code Prompt (Deliverables)

Below is a structured prompt for **Claude Code** to implement the backend. It specifies required output files, code style, and modules.

### Prompt Outline

You are to generate a full Node.js+TypeScript backend project. Follow these instructions exactly:

- Repository Structure:** Create a folder ``src/``. Inside, create modules: ``auth``, ``users``, ``vendors``, ``pets``, ``services``, ``bookings``, ``payments``, ``notifications``, ``chat``, ``uploads``, ``search``, ``analytics``. Also ``common`` for utilities. Provide subfolders:
  - ``controllers/``, ``services/``, ``repositories/``, ``routes/``, ``schemas/``, ``middlewares/`` for each module.
  - ``models/`` for Mongoose schemas.
  - ``config/`` for environment and config.
  - ``jobs/`` for background job definitions.
  - ``tests/`` for unit and integration tests.
- Tech and Style:** Use TypeScript with ``tsconfig.json`` set to ``strict: true``. Do NOT use ``any`` type. Prefer `async/await`. Use ES modules (``import``/``export``). Consistent code style (4-space indent, semicolons, etc.). Include ESLint/Prettier configs.
- Authentication:** Implement JWT auth in ``src/auth``. Files:
  - ``auth.controller.ts``, ``auth.service.ts``, ``auth.repository.ts``, ``auth.route.ts``, ``auth.schema.ts`` (Zod schemas for req/resp).
  - Use ``jsonwebtoken``. Implement signup (email/OTP), login, refresh token rotation, logout, password reset.
  - Refresh tokens stored in DB with user sessions (Mongoose model).
- User and Vendor Modules:**
  - ``users.controller.ts/services.ts/repository.ts/route.ts/schema.ts``. Same for ``vendors``.
  - Vendor signup should include KYC fields.
  - ``users.schema.ts`` and ``vendors.schema.ts`` include fields per design. Use

Zod for input validation.

5. **Pets Module:** ``pets.controller.ts``, etc. CRUD pets for users. Zod schemas for request bodies.
6. **Services Module:** Controllers/services for vendors to add/list services. Public listing route in ``services.route.ts`` for users.
7. **Bookings Module:** Controllers/services for creating and updating bookings. Implement state transitions. OTP logic (you may assume a helper service for sending OTPs).
  - ``booking.schema.ts`` for Zod.
  - Ensure booking creation checks availability (no overlap).
  - Use middleware to ensure only authorized roles can change status.
8. **Payments Module:** Integrate Razorpay using their Node SDK.
  - ``payments.controller.ts`` with endpoints for creating orders, handling webhooks.
  - ``payments.service.ts`` to interact with Razorpay.
  - Webhook signature verification.
  - ``payments.repository.ts`` for Payment model.
9. **Notifications Module:**
  - ``notifications.controller.ts/service.ts/repository.ts/route.ts/schema.ts``.
  - Endpoint to list and mark read.
  - Service can enqueue jobs to send FCM/email.
10. **Chat Module:**
  - ``chat.controller.ts/service.ts/repository.ts/route.ts``.
  - Models for ChatRoom and Message.
  - Use Socket.io (setup a ``chatGateway`` in main app).
  - Chat route can use both REST and WebSocket.
11. **Uploads Module:**
  - ``uploads.controller.ts`` with multer.
  - Use Cloudinary SDK to upload images.
  - Store file info in ``uploads.repository``.
12. **Search Module:**
  - Implement text search endpoints using Mongoose text index queries.
  - ``search.service.ts`` with ``model.find({ $text: { $search: query } })``.
13. **Analytics Module:**
  - Endpoints for admin reports (aggregate queries).
  - No heavy logic, just Mongoose aggregations.
14. **Middleware & Security:**
  - JWT auth middleware in ``common/jwt.middleware.ts``.
  - Role-check middleware ``common/role.middleware.ts``.
  - Validation middleware to use Zod schemas.

- Use ``helmet()`` and ``express-rate-limit`` at app level.
15. **\*\*Infrastructure:\*\***
    - ``Dockerfile`` (Node 18, install dependencies, build TS, expose port).
    - ``docker-compose.yml`` with services: ``app``, ``mongo``, ``redis``.
    - Provide ``.env.example`` with placeholders for secrets (JWT secret, DB URI, etc.)
  16. **\*\*Observability:\*\***
    - Setup Pino logger in ``app.ts``.
    - Expose ``/metrics`` for Prometheus (use ``prom-client``).
  17. **\*\*Testing:\*\***
    - Example test in ``tests/``, e.g. ``auth.test.ts`` using Vitest or Jest.
    - Mock DB with in-memory Mongo or jest mocks.
  18. **\*\*Documentation:\*\***
    - Generate OpenAPI/Swagger spec files. Use comments or ``swagger-jsdoc``.
    - Provide a sample Postman collection (JSON) if possible.
  19. **\*\*Checklist:\*\***
    - Each module has controller, service, repo, schema, route files (TS).
    - No lint errors, TS strict.
    - Sensitive data out of code (use ``process.env``).
    - Code is modular, commented, and idiomatic TypeScript.

Upon completion, ensure:

- All endpoints are defined with correct HTTP status codes and error handling.
- Zod schemas or JSON schemas for request/response validation.
- Database models match the provided data models.
- Role checks on protected routes.
- Docker image builds without errors.

Return code only (no explanation), with files and folder structure as output.

This completes the directive for Claude Code.

---

**Sources:** In crafting this design, we referenced industry best practices and documentation: authentication patterns (Auth0's refresh-token rotation [3†L503-L510] ), RBAC in Node.js [6†L144-L152] , MongoDB indexing strategies [10†L394-L402] [15†L1-L4] , Mongoose geospatial indexing [17†L211-L220] , API naming conventions [23†L169-L178] , logging with Pino [28†L91-L99] , OpenTelemetry observability [31†L858-L866] , and Express security best practices [43†L52-L59] [43†L76-L84] . These sources informed our recommendations for a robust, scalable backend.

---